

预备知识 P02：神经网络最小闭环——手写一个 MLP + 训练循环

P01 把张量和 autograd 这两件零件讲清楚了——但单独看张量、看一次 backward 都还只是"零件"，没看到"机器开起来什么样"。这一章把它们组装成训练时永远要走的三步：前向 → 反向 → 优化器。我们会从零写一个最小的 MLP（Multi-Layer Perceptron，多层感知机），在玩具数据上把它训练到收敛，并实时画出 loss 下降曲线。

所有训练相关任务（语言模型预训练、SFT、LoRA、RLHF 等）都共用这套「数据 → forward → loss → backward → step → zero_grad」的骨架——只是 forward 里的模型从 MLP 换成 Transformer，loss 从 MSE 换成 cross-entropy / KL，但循环结构本身一字不改。这一章打的就是这个骨架。

想直接跑示例？点这里  [Open in Colab](#)。

硬件门槛：概念章，CPU 即可✓。本章用一个 2 层 MLP 在 1000 个二维点上做二分类，参数总数不超过 200，CPU 训练一轮不到 1 秒。

一、MLP：最简单的可训练神经网络

MLP（多层感知机）是最简单的可训练神经网络——只有线性层 + 激活函数堆叠，没有 attention / 卷积 / RNN 这些更复杂的结构。优点：

- 结构最少：所有"为什么是这样"的问题（前向怎么算？loss 怎么求？反向怎么传？）都暴露在最少的代码里
- 训练最快：CPU 上几秒就能跑完一轮，迭代验证想法成本低
- 后续所有结构的子部件：Transformer 里的 FFN（feed-forward network）就是一个 2 层 MLP；attention 投影矩阵就是 `nn.Linear`；分类头就是一个线性层 + softmax

把 MLP 跑通，等于把"训练一个神经网络"这件事的最小可行版本走完一遍。

而且 MLP 不只是"教学玩具"——在真实系统里它本身就是高频出现的模块：

- Transformer 里的 FFN：每个 Transformer block 在 attention 之后都接一个 2 层 MLP（`Linear` → 激活 → `Linear`），这部分占了 LLM 总参数量的大头（通常 ~2/3）。换句话说，你用的每一个 LLM 里，绝大多数权重就是在做 MLP。
- 分类 / 回归头（head）：把 backbone（Transformer / CNN / ViT）输出的特征向量映射成最终预测——情感分类、token 分类、奖励模型打分（reward model 输出标量）、价值函数（critic 输出 V 值）几乎都是一个一两层 MLP。
- 嵌入投影（projection）：把不同来源的特征对齐到同一空间，例如多模态模型把视觉编码器输出的特征用 MLP 投影到语言模型的 embedding 维度；对比学习里也用 MLP projector 把表征映射到对比空间。
- MoE（Mixture of Experts）的专家：MoE 架构里每个 "expert" 通常就是一个独立的 FFN-MLP，路由器选若干个 MLP 来处理当前 token。

- 结构化 / 表格数据建模：在没有序列、没有空间结构的纯特征向量场景（推荐系统的特征交叉、风控模型、强化学习的策略 / 价值网络），MLP 至今仍是默认选择。

二、MLP 是什么

2.1 数学定义

一个 L 层 MLP 把输入向量 $x \in \mathbb{R}^{D_{\text{in}}}$ 一路变换成输出 $y \in \mathbb{R}^{D_{\text{out}}}$ ：

$$\begin{aligned} h_0 &= x \\ h_l &= \sigma(W_l h_{l-1} + b_l), \quad l = 1, 2, \dots, L-1 \\ y &= W_L h_{L-1} + b_L \end{aligned}$$

其中：

- $W_l \in \mathbb{R}^{D_l \times D_{l-1}}$ 是第 l 层的权重矩阵， $b_l \in \mathbb{R}^{D_l}$ 是偏置向量
- σ 是非线性激活函数（如 ReLU、sigmoid、GELU）。最后一层一般不加激活，让输出是任意实数（分类 logits / 回归值）
- D_l 叫第 l 层的隐藏维度（hidden dim）

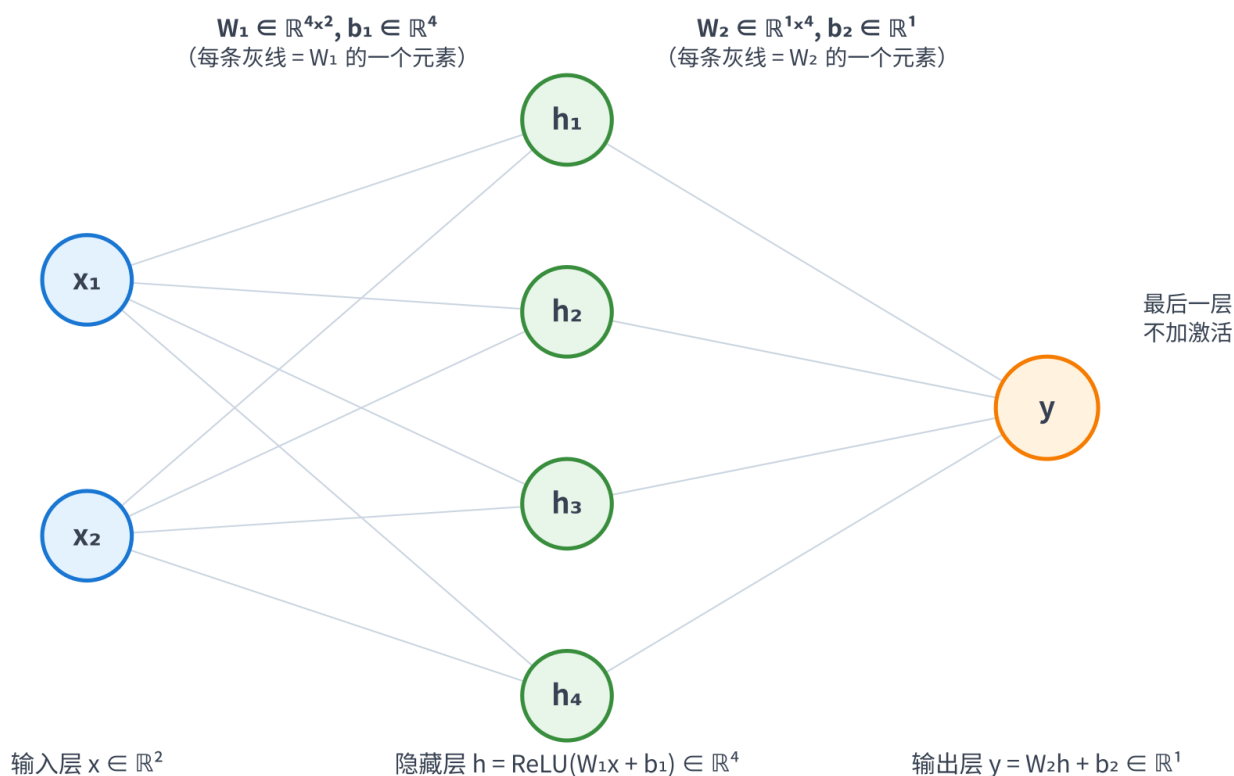
记号统一性提示：神经网络教材里有时用列向量、有时用行向量。PyTorch 的 `nn.Linear(in, out)` 内部是 `out = x @ W.T + b`（详见第 4.1 节）；下面公式按教科书惯例用列向量，但只要记住每一步的形状变换就不会出错。

最小例子（2 层 MLP）：输入 2 维 $\rightarrow$ 隐藏 4 维 $\rightarrow$ 输出 1 维（标量，二分类 logit）。

```
x ∈ ℝ²
|  W₁ shape (4, 2),  b₁ shape (4,)
▼
h = ReLU(W₁ x + b₁) ∈ ℝ⁴
|  W₂ shape (1, 4),  b₂ shape (1,)
▼
y = W₂ h + b₂          ∈ ℝ¹
```

把这个最小例子画成图——蓝色是输入层、绿色是隐藏层、橙色是输出层；每条灰线是一条全连接的权重连接（即 W_l 的一个元素），偏置 b_l 单独标注：

2 层 MLP 结构示意 (输入 2 维 → 隐藏 4 维 → 输出 1 维)



读图要点：

- 节点数 = 该层维度：输入 2 个圆 $\Rightarrow D_0 = 2$ ；隐藏 4 个圆 $\Rightarrow D_1 = 4$ ；输出 1 个圆 $\Rightarrow D_2 = 1$
- 权重在边上，偏置在节点上：从输入到隐藏一共 $4 \times 2 = 8$ 条边，正好对应 $W_1 \in \mathbb{R}^{4 \times 2}$ 的 8 个元素；每个隐藏节点还附带一个偏置 b_1 的元素，所以 $b_1 \in \mathbb{R}^4$
- 激活作用在节点上：隐藏层每个圆里实际做的是 $h_i = \text{ReLU}(\sum_j W_{1,ij} x_j + b_{1,i})$ ；输出层不加激活，只做线性组合

参数总数： $4 \times 2 + 4 + 1 \times 4 + 1 = 17$ 。

2.2 为什么必须有非线性激活

如果去掉 σ ，把多层线性变换叠起来：

$$y = W_L(W_{L-1}(\dots(W_1 x + b_1)\dots) + b_{L-1}) + b_L$$

展开后等价于 $y = W'x + b'$ ， $W' = W_L W_{L-1} \dots W_1$ ，整个网络退化成一个线性层——加多少层都一样。非线性激活是「让多层有意义」的关键。

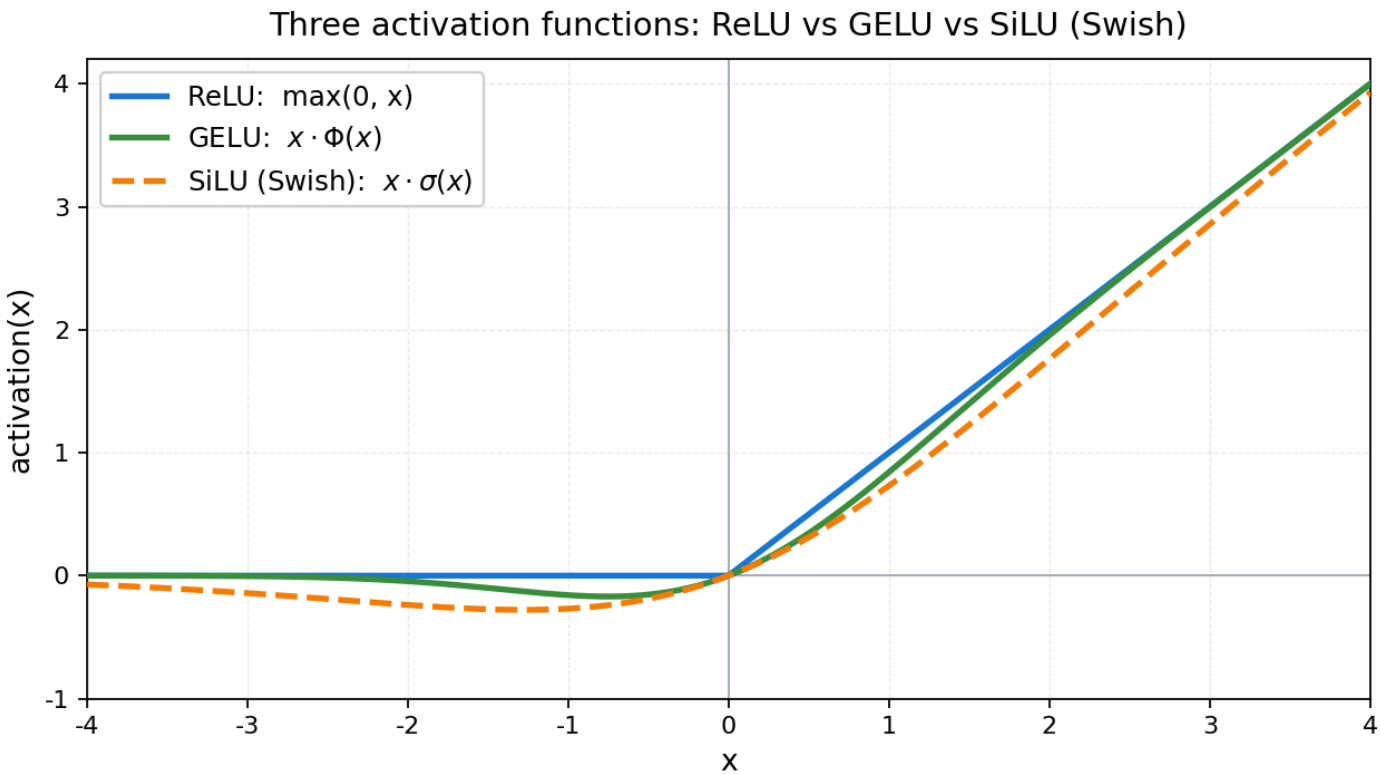
常见激活函数：

激活	公式	特点	LLM 里
ReLU	$\max(0, x)$	计算快，负值清零；可能"死亡"（永远输出 0）	老一代 Transformer FFN
GELU	$x \cdot \Phi(x)$ ， Φ 是标准正态 CDF	平滑、接近 ReLU 但保留少量负值信息	GPT-2/3 的 FFN
SwiGLU	由 SiLU（Swish）门控 + 线性组合而成	门控单元；同等参数量下表达能力更强	现代主流 LLM（LLaMA / Qwen / DeepSeek 等）的 FFN

缩写说明：

- FFN: Feed-Forward Network，前馈网络——指 Transformer block 中 attention 之后的 2 层 MLP（结构是 `Linear` → 激活 → `Linear`）。
- CDF: Cumulative Distribution Function，累积分布函数。 $\Phi(x) = P(X \leq x)$ ，其中 $X \sim \mathcal{N}(0,1)$ 是标准正态变量； $\Phi(x)$ 把 x 映射到 $(0,1)$ ，于是 GELU 可以理解为「ReLU 的平滑版本」——用 $\Phi(x)$ 这个软门控代替 ReLU 的硬阈值 $1[x > 0]$ 。

把表中三种激活的曲线画在同一张图上对比（SwiGLU 内部用两套权重 W_1, W_2 投影 + 门控，输出还依赖矩阵参数，没法像 ReLU/GELU 那样画成一条一维曲线；这里画的是它内部使用的非线性 SiLU / Swish—— $\text{SwiGLU}(x) = \text{SiLU}(W_1x) \odot (W_2x)$ ，曲线形状由 SiLU 决定）：



读图要点：

- 正半轴：三者都接近 $y = x$ ——大正值时几乎不衰减，能让梯度无障碍通过
- 负半轴：ReLU 直接归零（这就是「死亡 ReLU」的源头：一旦输入持续为负，梯度也是 0，权重永远更新不动）；GELU / SiLU 在 $x \in [-1.5, 0]$ 范围内留了一个"小坑"——GELU 最低点约 -0.170（在 $x \approx -0.75$ 处），SiLU 最低点约 -0.278（在 $x \approx -1.278$ 处），允许少量负信息和梯度漏过去
- 零点附近：ReLU 在 $x = 0$ 不可导（左导数 0、右导数 1）；GELU / SiLU 处处光滑——这对优化稳定性更友好，也是现代 LLM 普遍弃用 ReLU 的主要原因之一

SwiGLU 备注： $\text{SwiGLU}(x) = \text{SiLU}(W_1x) \odot (W_2x)$ 看似突兀，其实就三件事：

- 两套独立权重 $W_1, W_2 \in \mathbb{R}^{D_{\text{hidden}} \times D_{\text{in}}}$ ：把同一个输入 x 投影两次，得到两个同形状向量——一路当 gate（门控信号），另一路当 value（被门控的值）
- $\odot$ 是逐元素乘（Hadamard product）：两个同形状向量按位相乘，不是矩阵乘
- SiLU 只作用在 gate 这一路：把 gate 通道的值大体压到非负——但留有一个"小坑"，最低约 -0.278（出现在 $z \approx -1.278$ 处，见前面那条橙色虚线）。直观上：大正输入 $\approx$ 原样放行，强负输入压到接近 0，轻度负输入留一点"软门"漏过去。所以"通过比例"这个词只是大体直觉，不要把它误解成严格的 $(0, 1)$ 门——SwiGLU 里的 gate 上不封顶、下也允许小幅负值



举个 $D_{\text{hidden}} = 3$ 的最小例子。设两路投影后得到：

$$W_1x = [2.0, -1.0, 0.5], \quad W_2x = [1.0, 3.0, -2.0]$$

先算 gate—— $\text{SiLU}(z) = z \cdot \sigma(z) = z / (1 + e^{-z})$ ，逐元素套上去：

$$\begin{aligned} \text{SiLU}(W_1x) &= [2.0 \cdot \sigma(2.0), -1.0 \cdot \sigma(-1.0), 0.5 \cdot \sigma(0.5)] \\ &= [2.0 \times 0.881, -1.0 \times 0.269, 0.5 \times 0.622] \approx [1.76, -0.27, 0.31] \end{aligned}$$

再按位乘 value：

$$\text{SwiGLU}(x) \approx [1.76 \times 1.0, -0.27 \times 3.0, 0.31 \times -2.0] = [1.76, -0.81, -0.62]$$

注意第二个通道：value 是 3.0（很大），但 gate 落在 SiLU 的负半轴小坑里（ $W_1x = -1$ ， $\text{SiLU}(-1) \approx -0.27$ ），最终输出被缩到 -0.81——量级压到 value 的约 1/4，方向还反了。这就是"门控"的字面含义：用一路信号决定另一路每个通道的"放大倍数"（在 SwiGLU 里这个倍数大体非负、但允许小幅负值，所以严格说是"加权"而非传统意义的 $(0, 1)$ 门）。

补一句：真实 Transformer 里的 SwiGLU FFN 还有第 3 个矩阵 $W_3 \in \mathbb{R}^{D_{\text{model}} \times D_{\text{hidden}}}$ 把隐藏维度投影回模型维度：

$$\text{FFN}(x) = W_3(\text{SiLU}(W_1x) \odot (W_2x))$$

而原始 Transformer (Vaswani et al. 2017) 里 ReLU FFN、以及 GPT-2/3 把激活换成 GELU 之后的 FFN，都只有 2 个矩阵：

$$\text{FFN}_{\text{ReLU/GELU}}(x) = W_2 \cdot \sigma(W_1 x + b_1) + b_2$$

所以同等隐藏维度下，SwiGLU 比 ReLU/GELU FFN 多 ~50% 参数。但在实践中现代 LLM (LLaMA / Qwen / DeepSeek 等) 会把 SwiGLU 的 D_{hidden} 从惯例的 $4D_{\text{model}}$ 缩减到 $\frac{8}{3}D_{\text{model}}$ ，以保持总参数量与 ReLU/GELU FFN 持平（忽略 bias）：

FFN 类型	矩阵数	隐藏维度 D_{hidden}	参数量
ReLU / GELU FFN	2	$4D_{\text{model}}$	$8 D_{\text{model}}^2$
SwiGLU FFN (朴素)	3	$4D_{\text{model}}$	$12 D_{\text{model}}^2$ (+50%)
SwiGLU FFN (LLaMA 等实际用的)	3	$\frac{8}{3}D_{\text{model}}$	$8 D_{\text{model}}^2$ (持平)

本章 MLP 用最经典的 ReLU 起手——把训练循环跑通是首要目标，GELU / SwiGLU 等更复杂激活的取舍留到后面遇到再展开。

三、训练循环的三步

训练神经网络，每个 batch 永远走这三步——开头再加一次 `optimizer.zero_grad()` 清掉上 batch 留下的梯度：

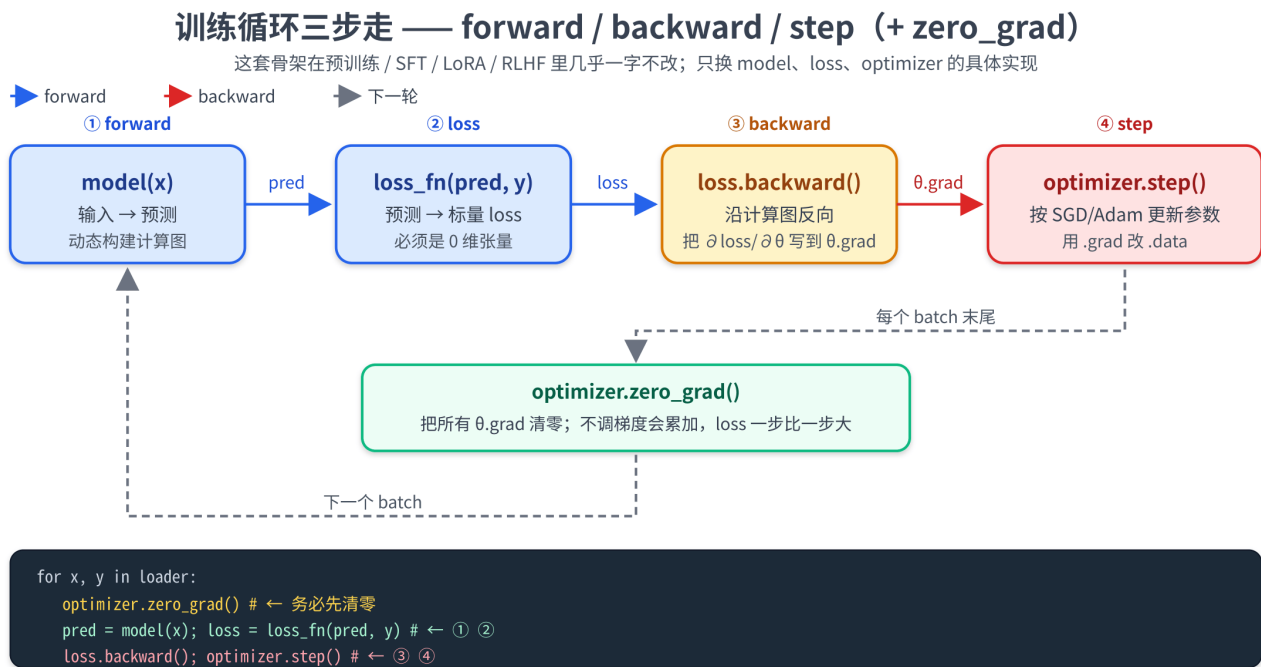


三步具体是：

- 1. 前向 (forward)：把 batch 喂进模型，算出预测和 loss。`pred = model(x); loss = loss_fn(pred, y)`
- 2. 反向 (backward)：`loss.backward()` 触发 autograd，把 $\partial \mathcal{L} / \partial \theta$ 累加到每个参数的 `.grad` 字段
- 3. 优化器更新 (step)：`optimizer.step()` 按某种规则 (SGD、Adam) 用 `.grad` 更新参数

而开头那次 `optimizer.zero_grad()` 是"清场"动作——把上一个 batch 留在每个参数 `.grad` 上的值清掉。PyTorch 的 `.grad` 是累加的，不主动清零会让多次 backward 的梯度相加、参数更新方向乱掉。

把这套循环画成图——上方一行是 forward 数据流（蓝色箭头）、再过 `loss.backward()` 把梯度回写到每个参数的 `.grad`（红色箭头）、然后 `optimizer.step()` 用 `.grad` 改 `.data`，每个 batch 末尾再回到开头继续：



这一节的所有内容、Transformer 的训练、LoRA 的训练、RLHF 的训练——骨架完全相同。区别只在 forward 里调什么模型、loss 怎么定义。

四、用 `nn.Module` 组织模型

PyTorch 用 `nn.Module` 这一抽象类组织一切"带参数的结构"——一层、一组层、整个模型都继承自它。

在写代码之前，先把第 2 节那些数学符号和这一节即将出现的 API 名字对一遍——后面所有对照都可以回到这张表上：

第 2 节里的理论概念	<code>nn.Module</code> 体系里对应的对象	形状 / 备注
权重矩阵 W_l	<code>nn.Linear(in, out).weight</code> (<code>nn.Parameter</code>)	(out, in) , 可训练
偏置向量 b_l	<code>nn.Linear(in, out).bias</code> (<code>nn.Parameter</code>)	(out,) , 可训练

第 2 节里的理论概念	<code>nn.Module</code> 体系里对应的对象	形状 / 备注
一次仿射变换 $W_l h_{l-1} + b_l$	<code>nn.Linear(in, out)</code> 整层	forward 时算 $xW^\top + b$, 详见第 4.1 节
激活函数 σ	<code>F.relu</code> / <code>F.gelu</code> 等函数式, 或 <code>nn.ReLU()</code> / <code>nn.GELU()</code> 等模块式	无参数, 不需要注册
中间隐藏向量 h_l	<code>forward</code> 内部的局部 tensor (如 <code>h = F.relu(...)</code>)	是 activation, 不是 <code>Parameter</code> , 反向完即丢
整网复合函数 $f_\theta(x)$	自定义类继承 <code>nn.Module</code> , <code>__init__</code> 注册子模块 + <code>forward</code> 串联	调用入口是 <code>model(x)</code>
全部可训练参数 $\theta = \{W_l, b_l\}_{l=1}^L$	<code>model.parameters()</code> / <code>model.named_parameters()</code>	喂给 optimizer 的就是它

API 层面的两条约定:

- `__init__` 里声明所有需要训练的子模块 (`self.fc1 = nn.Linear(...)`)
- `forward(self, x)` 里写前向计算流; 调用 `model(x)` 等价于 `model.forward(x)` , 但 PyTorch 会在外面套上 hook 等机制, 永远写 `model(x)`

下面就把第 2.1 节那个 2 层 MLP ($f_\theta(x)$ 在 $L = 2$ 时的具体形态) 按这两条约定写出来—— `fc1` / `fc2` 装的正是 W_1, b_1 和 W_2, b_2 这两组参数, `forward` 里的两行代码逐字对应公式 $h = \text{ReLU}(W_1 x + b_1)$ 和 $y = W_2 h + b_2$:

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class MLP(nn.Module):
    def __init__(self, in_dim, hidden_dim, out_dim):
        super().__init__()
        self.fc1 = nn.Linear(in_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, out_dim)

        def forward(self, x):
            h = F.relu(self.fc1(x))
            y = self.fc2(h)
            return y

model = MLP(in_dim=2, hidden_dim=4, out_dim=1)
print(model)
```

`print(model)` 会自动列出层级结构——这是 `nn.Module` 提供的"自我描述"能力。

4.1 nn.Linear 内部做了什么

```
linear = nn.Linear(in_features=2, out_features=4)
# linear.weight.shape == (4, 2)          ← (out, in)
# linear.bias.shape    == (4,)
```

nn.Linear 在 forward 时计算的是：

$$y = xW^T + b$$

注意是 W^T ——PyTorch 内部把 weight 存成 (out, in)（约定如此，许多深度学习框架都是这一布局：每一行对应一个输出神经元的输入权重）。所以 linear.weight.shape == (out, in)，乘的时候转置一下。这条容易记错：调试时 print([p.shape for p in linear.parameters()]) 一眼就能看清（注意要套一层列表推导——直接 print(p.shape for p in ...) 打出来的是一个 generator 对象，什么也看不到）。

nn.Linear 默认会自动初始化权重（Kaiming uniform）；想自定义初始化时再覆盖。

4.2 nn.Parameter 与 model.parameters()

nn.Linear 内部的 weight 和 bias 都是 nn.Parameter 类型——本质上是 requires_grad=True 的 tensor，并且自动注册到所属的 nn.Module，能通过 model.parameters() 一次性拿到全部需要训练的张量：

```
for name, p in model.named_parameters():
    print(name, p.shape, p.requires_grad)

# fc1.weight torch.Size([4, 2]) True
# fc1.bias   torch.Size([4])   True
# fc2.weight torch.Size([1, 4]) True
# fc2.bias   torch.Size([1])   True
```

下一步把 model.parameters() 交给 optimizer，就能让 optimizer 在 step() 里更新它们。

五、损失函数与优化器

5.1 损失函数

不同任务用不同 loss：

任务	损失	PyTorch API	备注
回归（输出连续值）	MSE（均方误差）	nn.MSELoss() / F.mse_loss	$\frac{1}{N} \sum (\hat{y} - y)^2$

任务	损失	PyTorch API	备注
二分类（每样本输出 1 个 logit）	BCE with logits	<code>nn.BCEWithLogitsLoss()</code> / <code>F.binary_cross_entropy_with_logits</code>	内部先 sigmoid 再 BCE，数值稳定
多分类（logits + 类别 id）	Cross-entropy	<code>nn.CrossEntropyLoss()</code> / <code>F.cross_entropy</code>	内部先 log_softmax 再 NLL，输入是 raw logits 不是 softmax 结果

表里那两条"内部先 ... 再 ..."值得展开讲一下——这是 PyTorch 在数值稳定性上做的标准融合，也是为什么传给它们的必须是 raw logits。

BCEWithLogitsLoss 把 sigmoid 和 BCE 熔在一起。朴素写法是

$$\mathcal{L} = -[y \log \sigma(z) + (1 - y) \log(1 - \sigma(z))], \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

其中 $z \in \mathbb{R}$ 是模型对这一个样本输出的 raw logit（任意实数，没有经过 sigmoid——在我们的 MLP 里就是 `fc2` 输出的那个标量）； $y \in \{0, 1\}$ 是这个样本的真实标签（0 = 负类，1 = 正类）； $\sigma(z) \in (0, 1)$ 是 sigmoid 把 logit 压成"预测为正类的概率"，所以 $1 - \sigma(z)$ 就是"预测为负类的概率"。整个 loss 的字面意思是：真实类别对应那一项概率的负对数—— $y = 1$ 时是 $-\log \sigma(z)$ ， $y = 0$ 时是 $-\log(1 - \sigma(z))$ ，预测越准 loss 越接近 0、预测越错 loss 越大。

当 logit z 很负时 $\sigma(z) \rightarrow 0$ 、 $\log \sigma(z) \rightarrow -\infty$ ；很正时 $1 - \sigma(z) \rightarrow 0$ 、 $\log(1 - \sigma(z)) \rightarrow -\infty$ 。朴素写法在浮点数上失败得比直觉早得多：正方向 fp32 在 $z \approx +17$ （fp16 约 +9）时 $\sigma(z)$ 就被舍入成恰好 1.0，于是 $\log(1 - \sigma(z)) = \log 0 = -\infty$ ——这一侧的问题不是 exp 上溢，而是"概率被舍入到 1"；负方向则要到 $z < -88$ （fp16 约 -11， e^{-z} 超过该 dtype 最大可表示数）才会上溢成 `inf` 把 $\sigma(z)$ 压成 0。一旦出现 $\log 0$ ，loss 先变 `inf`，再混进 $0 \times \infty$ 这类组合就成了 `nan`。融合版本改写成数值稳定的等价形式：

$$\mathcal{L} = \max(z, 0) - z \cdot y + \log(1 + e^{-|z|})$$

关键是 $e^{-|z|} \leq 1$ 不会上溢， $\log(1 + e^{-|z|})$ 始终落在 $(0, \log 2]$ ，再大的 $|z|$ 也不会让中间结果越界。所以传 raw logit 就行；手动先 `sigmoid` 再用 `BCELoss` 等于自愿退回那个不稳定的版本。

CrossEntropyLoss 把 log_softmax 和 NLL 熔在一起。朴素写法

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}, \quad \mathcal{L} = -\log p_c$$

其中 K 是类别数（多分类有几类，比如 LLM 的下一个 token 预测里 K 等于词表大小）； $z = (z_1, z_2, \dots, z_K) \in \mathbb{R}^K$ 是模型对这一个样本输出的 K 维 raw logits 向量（每一类一个实数，没经过 softmax）； z_i 是其中第 i 类的 logit； $\sum_j$ 表示对所有类别 $j = 1, 2, \dots, K$ 求和（分母是归一化常数）； $p_i \in (0, 1)$ 是 softmax 把 logits 压成概率分布后第 i 类的概率，满足 $\sum_i p_i = 1$ ； $c \in \{1, 2, \dots, K\}$ 是这个样本的

真实类别 id。整个 loss 的字面意思是：真实那一类 c 对应概率的负对数——预测准（ $p_c \rightarrow 1$ ）时 $\text{loss} \rightarrow 0$ ，预测错（ $p_c \rightarrow 0$ ）时 $\text{loss} \rightarrow +\infty$ 。

两处会数值越界—— $\sum_j e^{z_j}$ 一旦某个 z_j 超过 $\ln(\text{max})$ （ max 指当前 dtype 的最大可表示数，fp32 约 3.4×10^{38} 、fp16 约 6.5×10^4 ，对应 $\ln(\text{max})$ 约 88 / 11）就上溢成 `inf`；正确类别概率 p_c 下溢到 0 之后 $\log 0 = -\infty$ 。融合版本用 `log-sum-exp` 技巧，先减去最大值 $m = \max_j z_j$ （logits 向量 z 的所有 K 个分量里最大的那个值）：

$$\log p_i = z_i - m - \log \sum_j e^{z_j - m}$$

减完之后最大那一项的 $e^{z_j - m} = 1$ ，其余都在 $(0, 1]$ ， $\sum$ 既不会上溢、 $\log$ 也不会拿到 0。拿到所有类别的 $\log p_i$ 之后，loss 只是取第 c 个分量再取负——这正是 PyTorch 把它叫 `log_softmax + NLL`（Negative Log Likelihood，负对数似然）的字面含义。

同样地，`F.cross_entropy(logits, target)` 期望传入 raw logits——它内部自己 `log_softmax` 一次。手动 `softmax` 过再传不会报错，但 loss 已经不是真正的 NLL：它有一个抬高的下界（模型再笃定也降不到 0）、概率校准失效、接近 one-hot 时梯度被压扁。最隐蔽的是小任务上准确率往往照常上升、训练曲线看不出异常——这类 bug 靠肉眼盯曲线抓不到，调试时容易卡很久。

LLM 的“下一个 token 预测”训练用的就是 `F.cross_entropy`——每个位置输出一条长度等于词表大小（典型 $\sim 10^5$ ）的 raw logits 向量，target 是真实的 token id，按位置算 cross-entropy 再求平均。词表越大、logits 数值范围越宽，上面那个 `log-sum-exp` 数值技巧越关键。

顺带预告：本节出现的几个名字——`softmax` / `log_softmax` / `log-sum-exp` / `cross-entropy` / `NLL`——下一章 P03 会从概率与信息论视角再讲一遍。

5.2 优化器：SGD 与 Adam

优化器决定“拿着 grad，怎么更新参数”。最简单的是 SGD（随机梯度下降）：

$$\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} \mathcal{L}$$

其中 η 是学习率（learning rate, lr）。直觉：每次往负梯度方向走一小步。

Adam / AdamW 在 SGD 上叠了两个改良——分别维护每个参数的“梯度均值”和“梯度二阶矩”。

一阶矩（动量， m_t ）：

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

其中 $g_t = \nabla_{\theta} \mathcal{L}$ 是第 t 步的梯度， $\beta_1 \in [0, 1]$ 是衰减系数（PyTorch 默认 0.9）， $m_0 = 0$ 。 m_t 是历史梯度的指数加权平均（EMA），方向上比单步 g_t 更平滑——这就是“动量”的直觉：连续几步同向时累加加速，方向震荡时彼此抵消。

二阶矩（方差， v_t ）：

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

其中 β_2 默认 0.999, g_t^2 是逐元素平方, $v_0 = 0$ 。 v_t 估计每个参数最近一段梯度的"量级"——用来给每个参数自适应分配步长: 梯度长期大的方向走小步、长期小的方向走大步。

组合起来, 核心的参数更新公式大致是:

$$\theta \leftarrow \theta - \eta \cdot \frac{m_t}{\sqrt{v_t} + \epsilon}$$

($\epsilon \approx 10^{-8}$ 防止除以 0; 实际还有 bias correction $\hat{m}_t = m_t / (1 - \beta_1^t)$ 修正初期估计偏差等细节, 留到 P04 展开。)

LLM 训练几乎一律用 AdamW (Adam + weight decay 修正)。本章先用 SGD 走通流程, 优化器的更多细节、warmup + cosine 调度留到 P04 展开。

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
# 或: optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-2)
```

`optimizer` 拿到的是 `model.parameters()` 这个生成器——它会记录每个参数的引用, 在 `step()` 时直接修改它们的 `.data`。

六、完整训练循环骨架

把上面的零件拼成一个最小训练循环, 这段代码请记到肌肉记忆:

batch 是什么: 一个 batch 就是把若干条样本沿第 0 维堆在一起形成的张量——本章的例子

`x_batch.shape == (B, 2)` 表示一次把 B 条二维样本一起喂进模型。B 称为 batch size, 是显存与梯度噪声之间的折中: 太小梯度抖、GPU 利用率低; 太大显存吃不下; 典型取 32 ~ 1024。每个 batch 走完 "forward → loss → backward → step" 三步算一次参数更新; 把整个训练集过一遍叫一个 epoch。

```
import torch
import torch.nn as nn
import torch.nn.functional as F

# 1. 准备模型 / 损失 / 优化器
model = MLP(2, 16, 1)
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)

# 2. 训练循环
for epoch in range(num_epochs):
    for x_batch, y_batch in data_loader:
        # ---- 三步走 ----
        # (a) 前向
        logits = model(x_batch)
        loss = F.binary_cross_entropy_with_logits(logits.squeeze(-1), y_batch)

        # (b) 反向
        optimizer.zero_grad()
        # 清掉上一个 batch 留下的梯度
```

```

        loss.backward()                # 计算  $\partial loss / \partial \text{参数}$ 

        # (c) 更新
        optimizer.step()              # 按 SGD/Adam 规则修改参数

    print(f'epoch {epoch}, loss {loss.item():.4f}')

```

`zero_grad()` 放在 `backward()` 之前还是 `step()` 之后都可以——关键是不要漏掉。两种风格都常见：

- 风格 A（开头清零）：`zero_grad` → `forward` → `loss` → `backward` → `step`
- 风格 B（结尾清零）：`forward` → `loss` → `backward` → `step` → `zero_grad`

效果相同。本仓库统一用风格 A，因为它把“开始一个新 step”的语义放在了循环开头，更符合直觉。

七、实战：在 `make_moons` 数据上训练 MLP

我们用 `sklearn.datasets.make_moons` 生成两个交叉的半月形点云做二分类——这是经典的“线性不可分”玩具数据集，用来直观验证“非线性激活让 MLP 能学会非线性边界”。

数据准备：

```

from sklearn.datasets import make_moons

# n_samples=1000 个点; noise=0.2 让两类边界稍有交错
# random_state 固定 seed, 保证每次跑得到一样的数据
X_np, y_np = make_moons(n_samples=1000, noise=0.2, random_state=0)

# numpy → tensor; BCEWithLogitsLoss 要求 target 是 float (不是 long)
X = torch.tensor(X_np, dtype=torch.float32)
y = torch.tensor(y_np, dtype=torch.float32)

```

完整训练循环（每个 epoch 同时记录 `loss` 和 `acc` —— `acc` 即 `accuracy` / 准确率，定义是「预测正确的样本数 ÷ 总样本数」，二分类里就是 `(sigmoid(logit) > 0.5).float() == y` 求平均，取值 0 ~ 1，1 表示全部预测正确）：

```

import torch
import torch.nn as nn
import torch.nn.functional as F

torch.manual_seed(42)                # 让模型初始化、训练过程可复现

class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(2, 16)
        self.fc2 = nn.Linear(16, 1)
    def forward(self, x):
        return self.fc2(F.relu(self.fc1(x)))

model = MLP()

```

```

# SGD 是最朴素的优化器，先用它跑通流程；2-16-1 这个 MLP 在 SGD lr=0.1 + 200 epoch
# 大约收敛到 acc ≈ 0.87——这是"SGD lr=0.1 + 200 epoch"这套配置的优化瓶颈，
# 不是数据噪声的上限：换 AdamW 同样 200 epoch 能到 ≈0.95，优化器细节留到 P04
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)

losses = []
accs = []
for epoch in range(200):
    # ===== 三步走 =====
    # (a) 前向：把整个数据集当作一个 batch
    logits = model(X).squeeze(-1) # (N, 1) → (N,)
    # binary_cross_entropy_with_logits 比手写 sigmoid + BCE 数值稳定得多
    # — 它内部用 softplus 形式  $\log(1 + e^{-|z|})$  改写，避免  $e^{|z|}$  上溢（详见第 5.1 节）
    loss = F.binary_cross_entropy_with_logits(logits, y)

    # (b) 反向：清零梯度（绝不能漏！），再 backward
    optimizer.zero_grad()
    loss.backward()

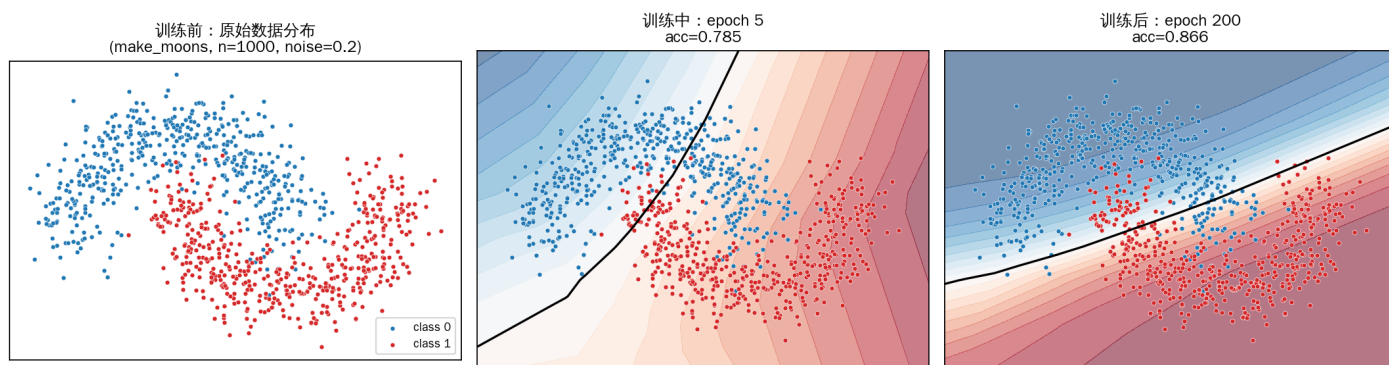
    # (c) 更新：optimizer 按 SGD 公式  $\theta \leftarrow \theta - lr * \theta.grad$  修改参数
    optimizer.step()

    # 记录 loss 与准确率，用于画曲线（注意用 detach()/no_grad 避免拉住计算图）
    losses.append(loss.item())
    with torch.no_grad():
        pred = (torch.sigmoid(logits) > 0.5).float()
        accs.append((pred == y).float().mean().item())

    if (epoch + 1) % 20 == 0:
        print(f'epoch {epoch+1:3d}  loss {loss.item():.4f}  acc {accs[-1]:.3f}')

```

把训练过程截成三张快照——训练前的原始数据、训练中（epoch 5）的边界、训练完（epoch 200）的边界——能直观看到 MLP 是怎么从"瞎猜"演化到"在两个半月之间挤出一条曲线"的：



读图要点：

- 训练前：原始 make_moons 数据，蓝点 = class 0、红点 = class 1，两个交叉的半月肉眼可见、但线性不可分
- 训练中（epoch 5）：模型刚动起来，决策边界接近一条斜直线、acc ≈ 0.79——MLP 还没学到"半月"的非线性几何，这一阶段它主要在调整 `fc2` 的偏置 + 大方向
- 训练后（epoch 200）：边界开始在两个半月之间弯曲，沿着两类的交界处贴合、acc ≈ 0.87——非线性激活让多层有意义的可视化证据。准确率没到 100% 是 `noise=0.2` 的设置让边界附近本就有交叉点，加上

SGD lr=0.1 200 epoch 在这个 2-16-1 的迷你 MLP 上大约就在 87% 左右收敛（换 AdamW 或加宽隐藏维度能再涨到 95%+，留给读者自行实验）

实战部分还会做两件事：

- 画训练 loss 曲线：让你看到 loss 单调下降——若不上升，多半是 lr 太大 / 太小，或者 `zero_grad` 漏了
- 画决策边界：把 2D 平面网格喂给训练好的 MLP，按预测的概率上色，能看到 MLP 在原本线性不可分的两个半月之间画出一条弯曲的非线性边界——直观证明"非线性激活让多层有意义"

并对比一个故意去掉 ReLU 的退化模型（`y = fc2(fc1(x))`）：会看到无论训多久，决策边界都只是一条直线（"两个线性层叠起来等价于一个线性层"在数据上的验证）。注意重点不是准确率——make_moons 上这个线性模型照样能到 $acc \approx 0.86$ ，和带 ReLU 的差不多；关键是它的边界永远是直线，没法贴合两个半月的弯曲交界。

八、常见踩坑

症状	原因	解决
loss 不下降，且数值稳定	漏调 <code>zero_grad()</code> 、 或 lr 太小	把 <code>zero_grad</code> / <code>backward</code> / <code>step</code> 三件套对齐
loss 突然变 NaN / inf	lr 太大、loss 内部 出现 <code>log(0)</code>	减小 lr；改用数值稳定的 loss API——把 raw logits 传给 <code>binary_cross_entropy_with_logits</code> 。注意 <code>binary_cross_entropy(sigmoid(x))</code> 不一定表现为 NaN（PyTorch 内部把 log 钳位在 -100），更多时候是 logit 一大梯度就饱和失真、训练悄悄变坏
<code>RuntimeError: ... not on the same device</code>	模型在 GPU 数据 在 CPU（或反过来）	在循环最外层 <code>model = model.to(device)</code> ，每 batch 取出后 <code>x = x.to(device)</code>
<code>RuntimeError: shape mismatch</code>	forward 里某一层 输入输出维度对不上	在每行后面 <code>print(x.shape)</code> ，从前往后对照
模型在训练集都拟合不了	模型容量太小 / 优化器选不对 / 输入未归一化	加宽 hidden_dim；换 AdamW；先把数据 mean-std 归一化
训练集 loss 下降但测试集很高	过拟合	加 dropout / weight_decay / 增数据；早停

九、关键概念回顾

概念	一句话定义	典型用途
MLP	线性层 + 非线性激活的堆叠	Transformer 内的 FFN 就是 2 层 MLP
<code>nn.Module</code>	组织"带参数结构"的基类，提供 <code>parameters()</code> / <code>state_dict()</code> 等	任何含可训练参数的代码都基于它
<code>nn.Linear</code>	实现 $y = xW^T + b$ ，weight shape 是 <code>(out, in)</code>	attention 的 Q/K/V 投影、输出投影、LoRA 低秩矩阵都是它
训练三步	forward → backward → step；外加 <code>zero_grad</code>	一切训练任务（预训练 / SFT / LoRA / RLHF）的循环骨架
损失函数	<code>F.cross_entropy</code> 输入是 raw logits，不是 softmax	语言模型的"下一个 token 预测"训练
优化器	SGD 是基线、AdamW 是 LLM 训练默认	优化器细节（动量、权重衰减、warmup）留到 P04 展开

十、本章小结

这一章把 P01 的零件组装成了能跑的"机器"：

- MLP 是最简的可训练神经网络——线性层 + 非线性激活的堆叠；非线性激活是"让多层有意义"的关键
- 用 `nn.Module` 组织模型——`__init__` 声明子模块、`forward` 写计算流；`nn.Linear(in, out)` 内部是 $y = xW^T + b$ ，weight shape 是 `(out, in)`
- 训练三步「forward → backward → step」+ 一个 `zero_grad` ——这套骨架在几乎所有训练任务里都不变
- 损失函数与优化器先掌握「`F.cross_entropy` 接收 raw logits」「先 SGD 跑通、再换 AdamW」两条；细节留到 P04
- 实战在 `make_moons` 上训练一个 2→16→1 的 MLP，观察到 loss 单调下降、决策边界从直线弯曲贴向两个半月之间——验证了"非线性激活 + 多层"能学会线性不可分

预告 P03：下一篇预备知识把训练里反复出现的概率工具（softmax、log-likelihood、熵、交叉熵、KL 散度、MLE）系统讲清楚。把 P01–P04 走完，再去读 attention 实现、写自己的训练脚本，几乎不会再有"卡在某个数学符号上"的时刻。